

三生万物：分支仿真引擎

三生万物：基于34行引擎的分支仿真引擎

作者：杨欣（独立研究者，中国）

2026年6月28日

摘要

本文介绍“三生万物”——一个基于道一34行最小心理涌现模型的分支仿真引擎。该引擎接收六道因果络分析框架的结构化输出，将历史事件转化为角色的记忆列表和资源池初始值，然后在道一引擎上运行多轮推演，预测未来走向。引擎支持三种仿真模式：基线推演（无干预自然演化）、事件推演（注入特定事件的分支if线）和干预推演（注入推荐方案的效果对比）。本文展示引擎的完整设计、World类多实体传播架构和效价方向判定机制。本文的核心主张是：一个极度简单的生成引擎，加载了足够丰富的记忆数据后，可以产生比复杂参数化系统更有意义的预测。

关键词：分支仿真、多实体传播、效价共振、效价冲突、世界引擎、情景推演

1. 引言

道一引擎用34行代码回答了一个问题：一个心理角色最少需要什么？答案是：一个资源池、一段记忆列表和一个tick函数。三个步骤——相似度比对、交叉传播、COR结算——足以在数千轮后涌现出敏感域、创伤簇和耗竭模式。

但道一引擎只处理一个角色面对一个事件的情况。现实世界是一个多实体系统：一个事件发生，不同实体因其不同的记忆而产生不同反应；一个实体的反应变成其他实体感知到的“次生事件”；次生事件再次触发各实体的记忆……这就是传播。

三生万物引擎的设计初衷就是回答这个传播问题：如果给多个实体各自加载历史事件的记忆，然后向这个系统注入一个新事件，会发生什么？谁的记忆与事件产生共振（被证实、被强化），谁的记忆与事件产生冲突（被否定、被打击）？谁赢谁输？

本文介绍三生万物引擎的完整设计。

2. 从分析框架到生成引擎

2.1 六道因果络提供了分析，但没有提供推演

六道因果络是一个通用社会系统分析元框架。它接收一个社会实体的历史数据，分析其六个维度的状态和18条因果路径的激活情况，输出一份结构化的诊断报告。但它只能分析“已经发生了什么”，不能推演“接下来会发生什么”。

三生万物填补了这个缺口。它接收六道因果络的输出——TR时间分片、六维度评分、路径激活状态——将这些数据转换为道一引擎的初始状态（memories列表 + pool初始值），然后运行多轮tick推演。

2.2 从参数化模拟到记忆驱动传播

三生万物的早期版本（v1-v6）是一个角色扮演对话引擎——每轮对话后运行一个参数化的后台计算，模拟角色的能量消耗、心之壁裂缝、崩塌触发。这个版本集成了五行生克的十元素矩阵、五域归因、十二长生阶段等传统概念。生产引擎文件（compute_state.py）最终达到了1472行。

但在道一34行引擎发现之后，三生万物经历了一次根本性的重构：所有参数映射被剥离，引擎的核心从“预设参数驱动的角色模拟”变为“记忆驱动的多实体传播仿真”。新的核心类不再是复杂的心理模拟器，而是一个简单的World类——管理多个道一Character实例，处理事件传播和次生事件生成。

2.3 五域归因与用神域：从预设到涌现

在早期版本中，每个角色的“五域敏感度”和“用神域”（最具恢复力的域）是通过配置文件预设的。在重构后的引擎中，这些概念被重新理解：五域不是预设的分类，而是记忆在嵌入空间中的聚类——记忆之间自然形成的簇自动成为事实上的“域”。用神域也不是预设的属性——它是角色在那些记忆效价最高的方向（最正面的记忆簇所在的区域）自然涌现的恢复力。

但这不意味着五域归因完全从工程中消失了。当需要通过配置文件精确控制角色行为时，域特异性敏感度仍然作为可选参数存在。这是“眼镜与世界”关系的具体体现：分析框架的语言（五域）在工程层面作为观测接口保留，但它们不再被视为引擎的必要零件。

3. 引擎架构

3.1 底层：道一34行引擎

三生万物引擎的每一轮tick都由道一的Character类驱动。该类的完整设计在道一篇中已详细阐述，此处仅概述核心逻辑：

每个Character实例包含： - pool：[0, 1]区间的资源池 - memories：[{"emb": 嵌入向量,

“intensity”: 冲击强度}, ...]

每轮tick的三步操作： 1. 遍历所有记忆，计算当前事件嵌入与每条记忆的余弦相似度。超过动态阈值（ $0.6 - \text{intensity} \times 0.4$ ）的记忆被激活。 2. 激活记忆之间计算交叉传播消耗。 3. COR结算： `pool = pool - drain + recovery`。

3.2 WorldCharacter: 效价方向判定

道一的34行引擎是为1对1对话设计的——每轮匹配0-2条记忆。在世界仿真中，一个宏观事件可能同时触动多条记忆。更关键的是，在人际对话中，“记忆被激活”几乎总是意味着额外的消耗（被触痛）。但在多实体仿真中，一个事件可能“证实”某个实体的记忆——这非但不是消耗，反而是恢复。

这就是 WorldCharacter 的核心修正。它继承自 Character，重写 tick() 方法，引入效价方向判定：

```
class WorldCharacter(Character):
    def tick(self, event_emb, intensity, base_drain=0.03, base_recovery=0.01):
        event_valence = event_emb[0] # 嵌入向量的第一维 = 效价
        drain = base_drain
        recovery = base_recovery
        matched_drain = [] # 反向冲突 → 消耗
        matched_recovery = [] # 同向共鸣 → 恢复

        for m in self.memories:
            sim = cosine_sim(event_emb, m["emb"])
            threshold = 0.7 - m["intensity"] * 0.3
            if sim > threshold:
                memory_valence = m["emb"][0]
                valence_agreement = 1.0 - abs(event_valence - memory_valence)
                # valence_agreement: 0=完全相反, 1=完全相同

                if valence_agreement >= 0.70:
                    # 同向共鸣 → 恢复
                    recovery += m["intensity"] * sim * valence_agreement * 0.06
                    matched_recovery.append(m)
                else:
                    # 反向冲突 → 消耗
                    drain += m["intensity"] * sim * (1.0 - valence_agreement) * 0.12
                    matched_drain.append(m)
```

效价共振（Valence Resonance）：当事件的效价方向（正面/负面）与某条记忆的效价方向一致（`valence_agreement ≥ 0.70`），该记忆不是被“触痛”而是被“证实”。被证实的高强度记忆产生恢复而非消耗。

效价冲突（Valence Conflict）：当事件的效价方向与记忆相反，该记忆被“否定”。被否定

的高强度记忆产生比普通激活更大的消耗。

交叉匹配也分方向：共振的记忆之间互相增强恢复，冲突的记忆之间互相加剧消耗。这确保了同向胜利被放大，同向失败被叠加。

参数差异：WorldCharacter的基础消耗（0.03）和基础恢复（0.01）比原版（0.05/0.02）更小，因为世界仿真的tick频率更高。激活阈值的基准从0.6提升到0.7，强度影响系数从0.4降至0.3——世界级事件通常涉及大量记忆，提高阈值避免“什么都激活”。

3.3 World类：多实体传播

```
class World:
    def __init__(self):
        self.entities: dict[str, Character] = {}
        self.history: list[dict] = []
        self.turn = 0

    def add_entity(self, name, pool, memories):
        self.entities[name] = WorldCharacter(pool=pool, memories=memories)

    def propagate(self, event_emb, intensity, event_desc=""):
        """一个事件打进所有entities。"""
        self.turn += 1
        results = {}
        for name, entity in self.entities.items():
            pool_before = entity.pool
            pool_after, matched, recorded, drain, recovery = entity.tick(event_emb, intensity)
            results[name] = {
                "pool_before": pool_before,
                "pool_after": pool_after,
                "delta": pool_after - pool_before,
                "resonance": [m for m in matched if m["_valence_effect"] == "resonance"],
                "conflict": [m for m in matched if m["_valence_effect"] == "conflict"],
            }
        self.history.append({"turn": self.turn, "event": event_desc, "results": results})
        return results
```

World类的核心设计原则是：**复杂的是每个entity的memories，不是World类**。World类只是一个容器——事件打进来，每个entity按自己的记忆独立反应，World记录所有人的反应。它不需要知道“域”“人格”“心之壁”这些概念——每个entity的道一Character实例自己处理一切。

3.4 次生事件传播

在多实体系统中，一个实体的剧烈状态变化本身就是一个事件。如果Entity A的pool从0.78暴跌到0.49，这个变化会被市场感知为“危机信号”，扩散到其他实体。

引擎实现了次生事件生成器：当某实体的delta超过阈值（ >0.02 为恢复信号， <-0.05 为危机信号），引擎自动生成一个描述该变化的事件，编码为新的嵌入向量和强度，再次打入所有实体。为避免无限级联，仅传播delta绝对值最大的前2个实体。

3.5 三种仿真模式

引擎支持三种推演模式：

模式1：基线推演。无外部干预，纯跑道一引擎的tick()循环N轮。每轮的事件由六道因果络TR分片的自然序列驱动。输出pool状态曲线、关键转折点和风险预警。

模式2：事件推演（What-If）。从基线的关键节点分叉，为每个if线创建一个独立的Character实例（克隆该节点的状态），注入特定事件。并排对比各分支线的pool轨迹。

模式3：干预推演。从风险预警点分支，注入六道因果络推荐的干预方案。每个方案对应一条分支线。输出干预效果对比和性价比分析。

5. 讨论

5.1 为什么一个极简引擎可以跑出这些

因为记忆不是引擎的“参数”——记忆是引擎的“数据”。34行代码定义了universal的生成规则（相似度比对→激活→交叉传播→结算），而每个实体的独特性完全来自它的记忆列表。如果China的五条记忆全部建立在“效率”叙事上，而USA的五条全部建立在“规模”叙事上，那么同一个事件（证明效率可行、规模非必要）自然会在这两个实体中产生完全相反的动力学——不需要任何“国家差异”参数。

5.2 与道一引擎、六道因果络的关系

三个引擎的分工： - **六道因果络**：分析——从历史数据中提取TR分片、维度评分、路径激活状态、风险点和方案。 - **三生万物**：推演——接收分析数据，加载为道一Character的初始状态，运行仿真预测。 - **道一**：运行——34行引擎为三生万物的每一轮tick提供底层计算。

5.3 引擎局限

嵌入向量的主观性。仿真结果的质量取决于事件嵌入和记忆嵌入的编码质量。嵌入向量的质量直接影响仿真结果，当使用启发式编码而非LLM语义嵌入时，主观性不可忽略。

次生事件生成的启发式。次生事件的嵌入由启发式规则生成——基于delta的符号和幅度。在理想情况下，次生事件的嵌入应由LLM根据上下文生成。

传播深度限制。为避免无限级联，当前引擎只传播delta最大的前2个实体。在真实系统中，

每个实体的变化都可能产生信号。

6. 结论

本文介绍了三生万物分支仿真引擎的设计和架构。该引擎基于道一的34行最小心理涌现模型，扩展了效价方向判定和World多实体传播机制，能够在记忆驱动的框架下推演多实体系统对一个事件的反应和后续传播。

三生万物的核心主张是：预测不需要复杂的参数化模型。一个极度简单的生成引擎，加载了足够丰富的历史记忆数据后，可以产生比复杂参数化系统更有意义的推演。

参考文献

1. Bakker, A. B., & Demerouti, E. (2007). The Job Demands-Resources model. *Journal of Managerial Psychology*, 22(3), 309–328.
2. Collins, R. (2004). *Interaction Ritual Chains*. Princeton University Press.
3. Hobfoll, S. E. (1989). Conservation of resources. *American Psychologist*, 44(3), 513–524.
4. 老子. (约公元前6世纪). 《道德经》. 第四十二章.

道一引擎完整设计见道一篇《道生万物：34行最小心理涌现模型》。六道因果络完整设计见六道因果络篇《六道因果络：通用社会系统分析元框架》。引擎源代码见 `world_engine.py`。

完成于2026年6月28日。基于`world_engine.py`和道一34行引擎。